MINISTRY OF EDUCATION, CULTURE AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Report

on laboratory work no. 3

of Cryptography and Security

Topic: Polyalphabetic Ciphers. The Playfair Cipher

Done by: st. gr. FAF-243, Augustin Carazanu
Checked by: *assist. univ. Maia Zaica*
FCIM, UTM

Chișinău – 2026

Purpose of the work

The purpose of this laboratory work is to study the Playfair cipher, one of the first ciphers that encrypts pairs of letters instead of single letters, and to implement it for messages in Romanian (31 letters). According to the lab sheet, students with an odd number in the group register solve Task 3.1 (Playfair). The program was written in Python 3 with a simple Tkinter interface: the user chooses encryption or decryption, enters the key and the message or the cryptogram, and gets the result.

1. Short theory

1.1. History

The algorithm was invented by Charles Wheatstone and described for the first time on 26 March 1854, but it carries the name of Baron Lyon Playfair, who convinced the British government to adopt it. It was used by the British army in the Boer War and, in modified versions, in both World Wars. Today it is easy to break, but it is one of the first algorithms that use the principle of modern block ciphers: the plaintext is processed in blocks of two letters.

1.2. The algorithm

Playfair encryption has three steps:

a) Preparing the text. All letters are written in uppercase, without spaces and punctuation, and the text is split into pairs. If both letters of a pair are equal, a rare filler letter is inserted between them, and if the last pair has only one letter, a filler is added at the end.

b) Building the matrix. Repeated letters are removed from the key, starting from the second occurrence. The matrix is filled row by row, first with the letters of the key and then with the remaining letters of the alphabet, in alphabetical order.

c) Encrypting the pairs. Each pair is encrypted with three rules:

1. letters in different rows and columns: each letter is replaced by the letter on its own row and in the column of the other letter (the corners of the rectangle);

2. letters on the same row: each letter is replaced by the next letter on the row; the last letter of the row is replaced by the first one;

3. letters in the same column: each letter is replaced by the letter right below it; the last letter of the column is replaced by the first one.

Decryption uses the same matrix and the same pairs. The rectangle rule stays the same, while on a row each letter is replaced by the previous one and in a column by the one above it.

1.3. Adapting the algorithm to the Romanian alphabet

The Romanian alphabet has 31 letters (A Ă Â B C D E F G H I Î J K L M N O P Q R S Ș T Ț U V W X Y Z), so a 5 × 5 matrix is not enough. The lab sheet allows other sizes (6 × 5, 5 × 6, 6 × 4, …) and allows replacing a rare letter, as J is replaced by I in the classic algorithm. In this implementation:

• the matrix has **5 rows and 6 columns (30 cells)**;

• the rare letter **J is replaced by I**, so the other 30 letters fill the matrix exactly and there are no empty cells or extra symbols;

• the filler letter is **X**, which is very rare in Romanian; if the letter that has to be separated is X itself, **Q** is used instead.

For the key SECURITATE, the letters without repetitions are S, E, C, U, R, I, T, A, and the matrix is:

Table 1.1 – The 5 × 6 encryption matrix for the key SECURITATE (the letters of the key are shaded)

S	E	C	U	R	I
T	A	Ă	Â	B	D
F	G	H	Î	K	L
M	N	O	P	Q	Ș
Ț	V	W	X	Y	Z

2. Implementation

The program has two files: `playfair.py` contains the algorithm and the input validation, and `gui.py` contains the Tkinter window. The window shows the encryption matrix for the entered key, the prepared pairs and the result. The main fragments are presented below.

2.1. The alphabet and the matrix size

```
ALPHABET = ["A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "J", "K", "L",
            "M", "N", "O", "P", "Q", "R", "S", "Ș", "T", "Ț", "U", "V", "W", "X", "Y",
"Z"]
LOWER = ["a", "ă", "â", "b", "c", "d", "e", "f", "g", "h", "i", "î", "j", "k", "l",
         "m", "n", "o", "p", "q", "r", "s", "ș", "t", "ț", "u", "v", "w", "x", "y",
"z"]

ROWS, COLS = 5, 6
MATRIX_LETTERS = [ch for ch in ALPHABET if ch != "J"]   # 30 letters, J -> I
FILLER = "X"            # inserted between double letters and at the end of an odd
text
FILLER_FOR_X = "Q"      # used when the letter to separate is X itself
```

The alphabet is stored in the order of the Romanian alphabet. `MATRIX_LETTERS` is the alphabet without J, which gives exactly 30 letters for the 5 × 6 matrix.

2.2. Input validation

```
def normalize(text, allow_spaces=True):
    """Uppercase letters only, spaces removed, J replaced by I."""
    out = []
    for ch in text:
        if ch in " \n\t" and allow_spaces:
            continue
        if ch not in TO_UPPER:
            raise ValueError(f"Invalid character '{ch}'. Allowed characters:
{ALLOWED}.")
        up = TO_UPPER[ch]
        out.append("I" if up == "J" else up)
    if not out:
        raise ValueError("The text must contain at least one letter.")
    return "".join(out)


def parse_key(key):
    """The key must have at least 7 letters and only allowed characters."""
    k = normalize(key)
    if len(k) < 7:
        raise ValueError(f"The key is too short ({len(k)} letters). It must have at
least 7 letters.")
    return k
```

`normalize()` removes the spaces, converts the text to uppercase, replaces J with I and rejects any other character, telling the user which characters are allowed. The cedilla forms Ş and Ţ are accepted as Ș and Ț. `parse_key()` also checks that the key has at least 7 letters.

2.3. Preparing the text

```
def filler_for(letter):
    return FILLER_FOR_X if letter == FILLER else FILLER


def prepare(text):
    """Split the text into pairs; a filler separates two equal letters
    of the same pair and completes a last single letter."""
    pairs, i = [], 0
    while i < len(text):
        a = text[i]
        b = text[i + 1] if i + 1 < len(text) else None
        if b is None:
            pairs.append(a + filler_for(a))
            i += 1
        elif a == b:
            pairs.append(a + filler_for(a))
            i += 1
        else:
            pairs.append(a + b)
            i += 2
    return pairs
```

`prepare()` goes through the text and forms the pairs. When the two letters of a pair would be equal, the filler is inserted after the first letter and the second letter moves to the next pair. A single last letter is completed with the filler.

2.4. Building the matrix

```
def unique_letters(key):
    """Remove repeated letters of the key, starting from the second occurrence."""
    seen = []
    for ch in key:
        if ch not in seen:
            seen.append(ch)
    return "".join(seen)


def build_matrix(key):
    order = unique_letters(key) + "".join(MATRIX_LETTERS)
    cells = unique_letters(order)
    return [list(cells[r * COLS:(r + 1) * COLS]) for r in range(ROWS)]


def positions(matrix):
    return {matrix[r][c]: (r, c) for r in range(ROWS) for c in range(COLS)}
```

`unique_letters()` keeps only the first occurrence of each letter. The matrix is obtained by writing the unique letters of the key followed by all the matrix letters, again without repetitions, and cutting this sequence into 5 rows of 6 letters.

2.5. Encrypting and decrypting a pair

```
def transform_pair(pair, matrix, pos, step):
    """step = +1 for encryption, -1 for decryption."""
    (r1, c1), (r2, c2) = pos[pair[0]], pos[pair[1]]
    if r1 == r2:                                    # same row: next / previous letter
in the row
        return matrix[r1][(c1 + step) % COLS] + matrix[r2][(c2 + step) % COLS]
    if c1 == c2:                                    # same column: letter below /
above
        return matrix[(r1 + step) % ROWS][c1] + matrix[(r2 + step) % ROWS][c2]
    return matrix[r1][c2] + matrix[r2][c1]          # rectangle: same row, other
column


def encrypt(message, key):
    matrix = build_matrix(parse_key(key))
    pos = positions(matrix)
    pairs = prepare(normalize(message))
    return matrix, pairs, [transform_pair(p, matrix, pos, +1) for p in pairs]


def decrypt(cryptogram, key):
    matrix = build_matrix(parse_key(key))
    pos = positions(matrix)
    text = normalize(cryptogram)
    if len(text) % 2:
        raise ValueError(f"A cryptogram must have an even number of letters (it has
{len(text)}).")
    pairs = [text[i:i + 2] for i in range(0, len(text), 2)]
    for p in pairs:
        if p[0] == p[1]:
            raise ValueError(f"The pair '{p}' has two equal letters, so it cannot come
from Playfair encryption.")
    return matrix, pairs, [transform_pair(p, matrix, pos, -1) for p in pairs]
```

`transform_pair()` implements the three rules. The parameter `step` is +1 for encryption (next letter on the row, letter below in the column) and −1 for decryption (previous letter, letter above); the modulo operations make the rows and columns circular. Before decryption the program checks that the cryptogram has an even number of letters and that no pair contains two equal letters, because such a text cannot be produced by Playfair encryption.

3. Results

The message “Criptografia este știința scrierii secrete” was encrypted with the key “securitate”. After removing the spaces, the text is split into pairs; the double I of “scrierii” is separated with X:

```
CR IP TO GR AF IA ES TE ȘT IX
IN ȚA SC RI ER IX IS EC RE TE
```

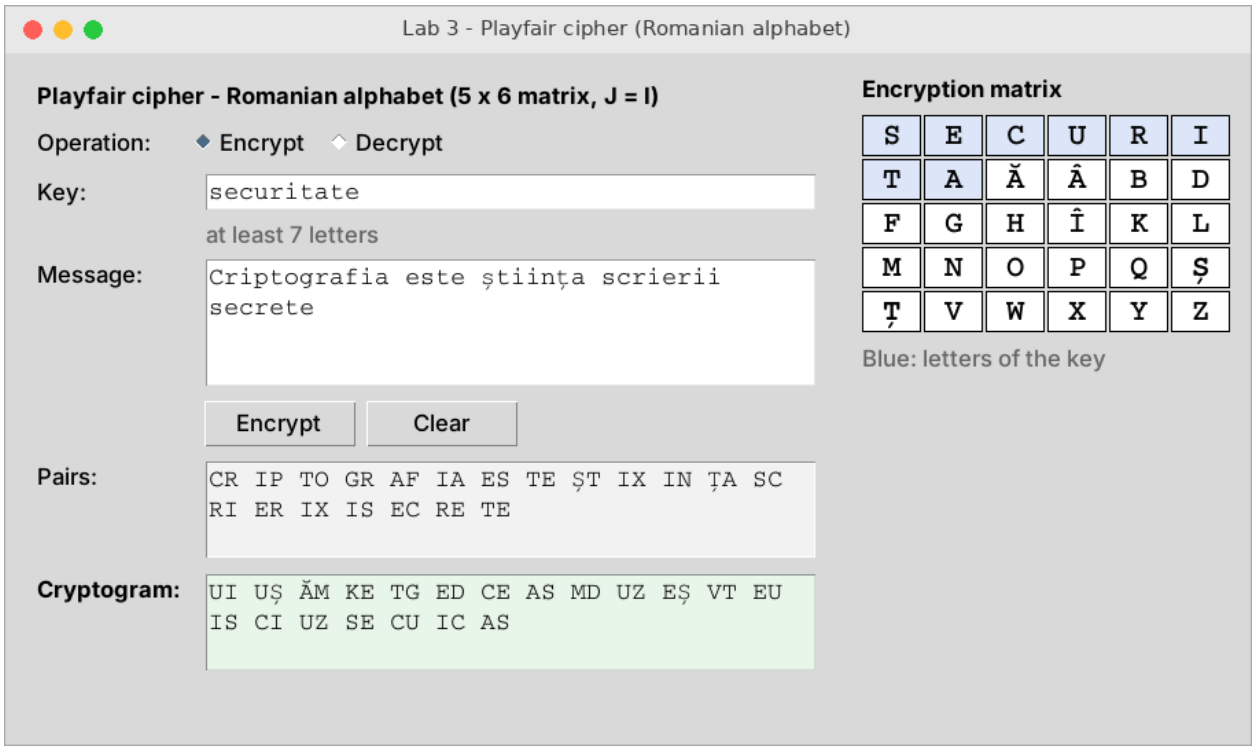


Figure 3.1 – Encryption with the key “securitate”

Manual check with the matrix from Table 1.1:

• CR – C and R are on the same row, so each is replaced by the next letter on the row: C → U, R → I, giving UI;

• IP – I (row 1, column 6) and P (row 4, column 4) form a rectangle: I → U, P → Ș, giving UȘ;

• TO – T (row 2, column 1) and O (row 4, column 3) form a rectangle: T → Ă, O → M, giving ĂM.

The program gives the same pairs.

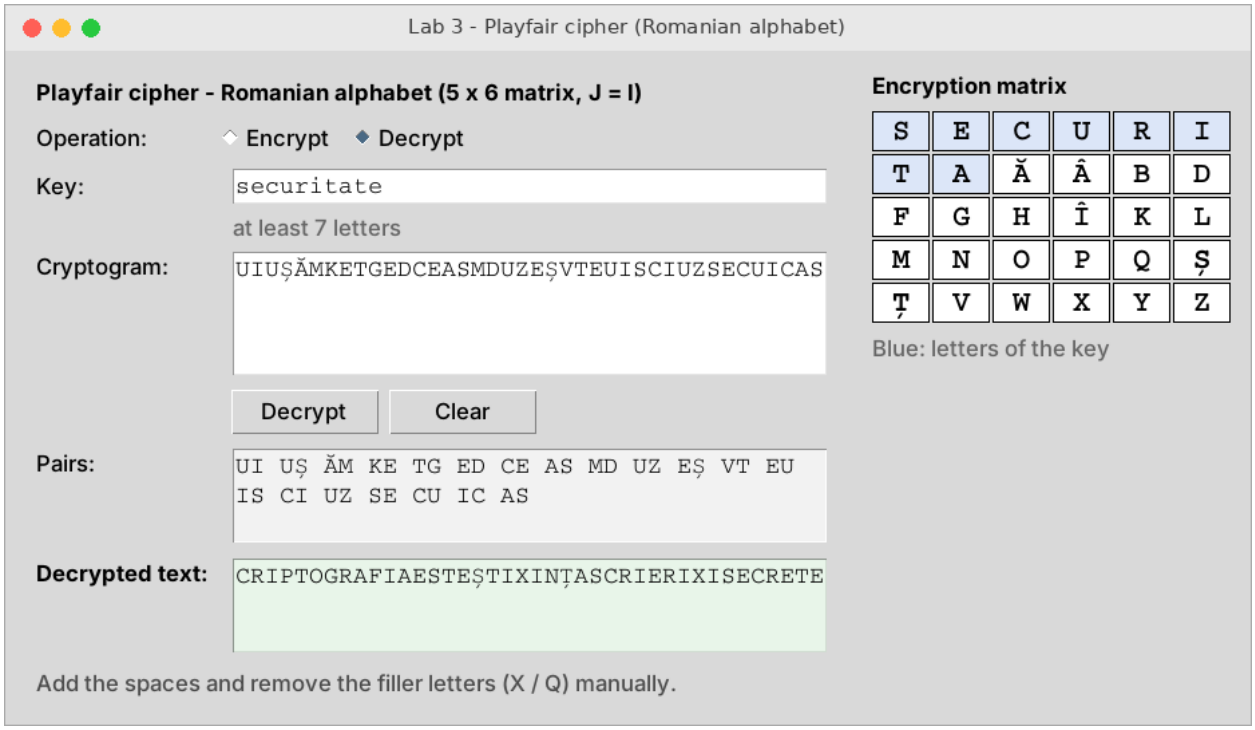


Figure 3.2 – Decryption of the cryptogram with the same key

Decryption gives back the prepared text `CRIPTOGRAFIAESTEȘTIXINȚASCRIERIXISECRETE`. As required by the lab sheet, the spaces and the filler letters are added and removed manually: CRIPTOGRAFIA ESTE ȘTI(X)INȚA SCRIERII SECRETE. The X inside “ȘTIINȚA” separates the double I, and the one inside “SCRIERII” too.

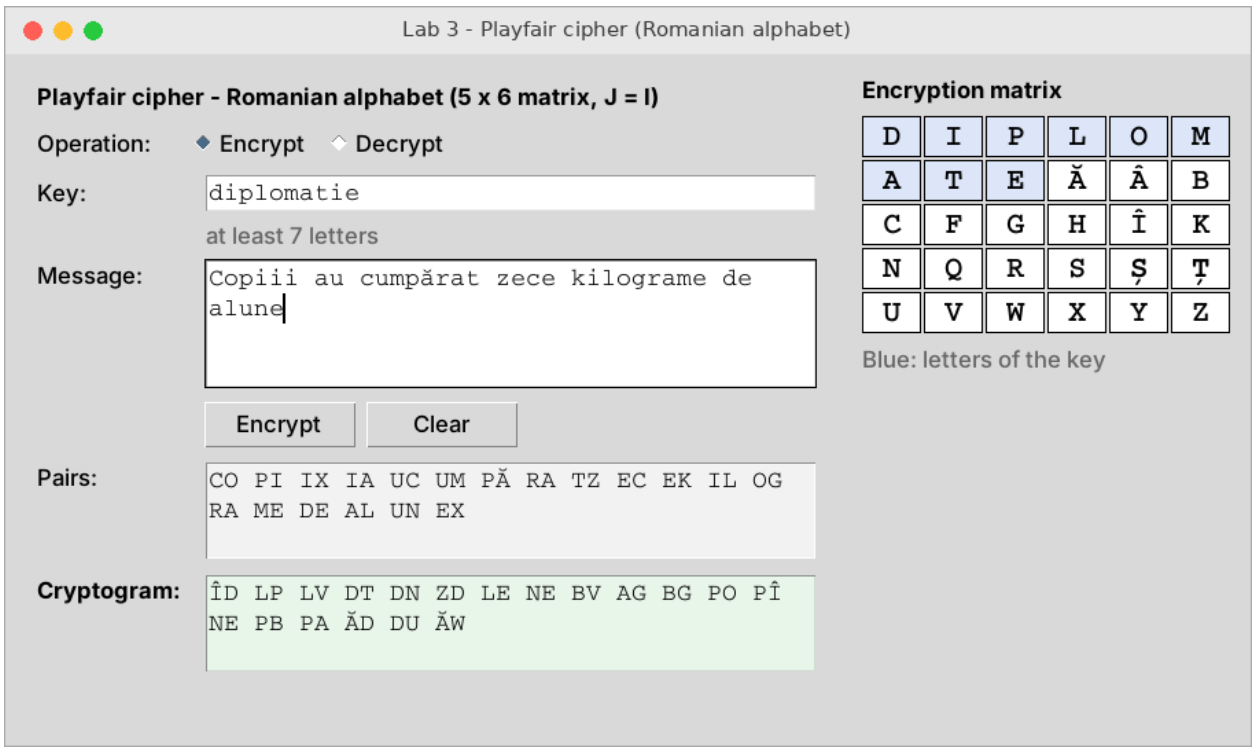


Figure 3.3 – A message with three equal letters (“Copiii”) and the key “diplomatie”

In “Copiii” the program forms the pairs CO PI IX IA: the second and third I are separated with X, and the third I goes into the next pair together with the A of “au”. The odd final letter is completed with X (EX).

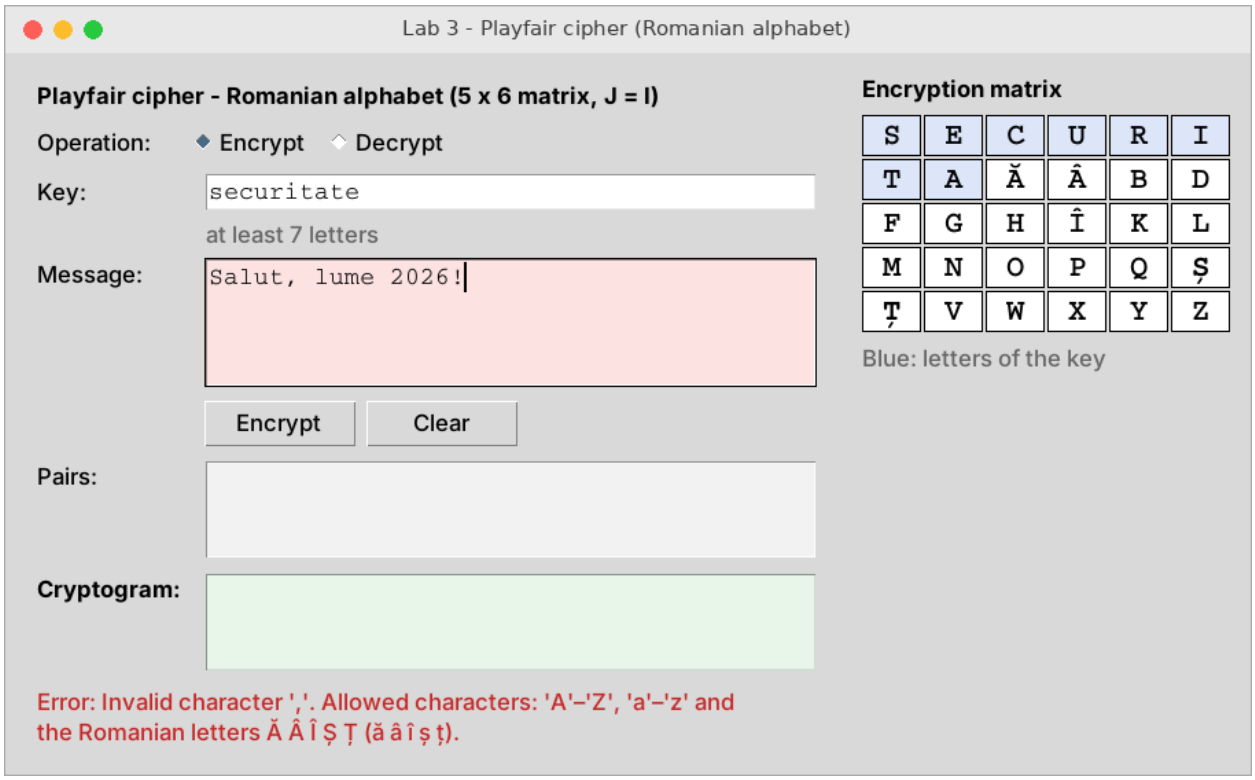


Figure 3.4 – A message that contains a comma, digits and an exclamation mark is rejected

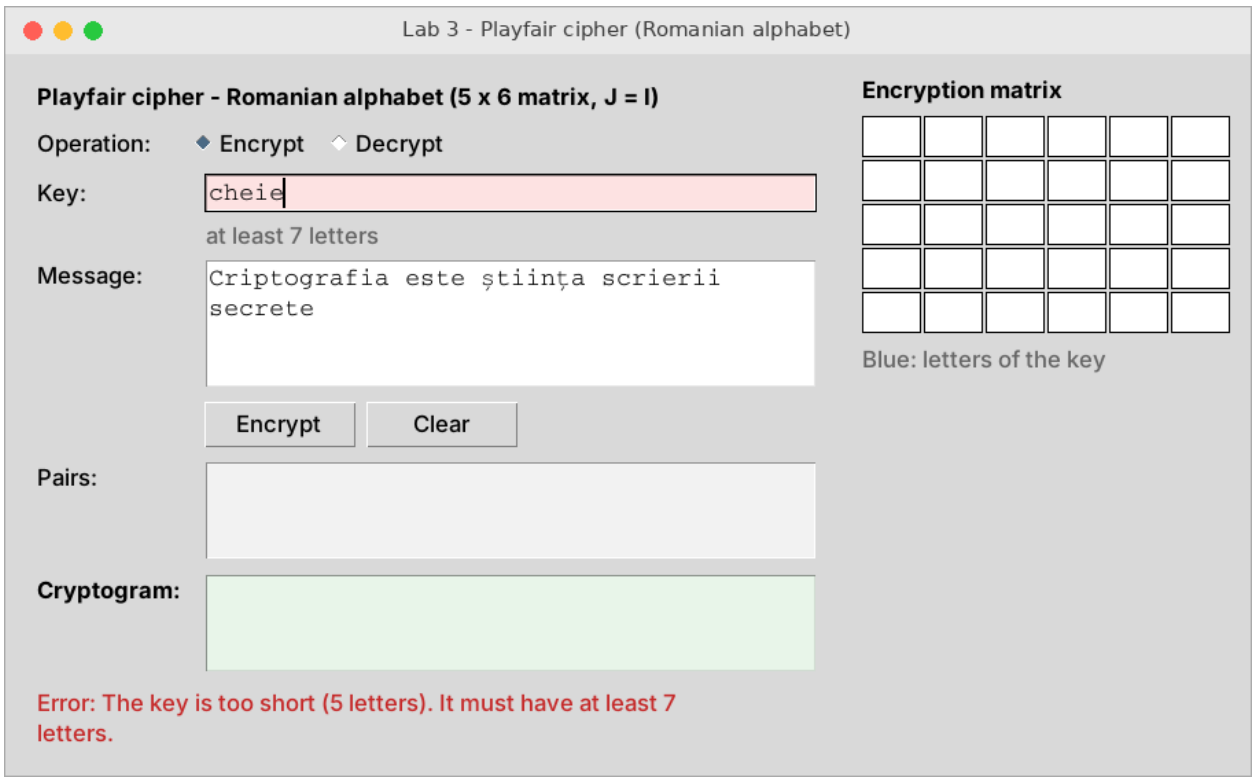


Figure 3.5 – A key shorter than 7 letters is rejected

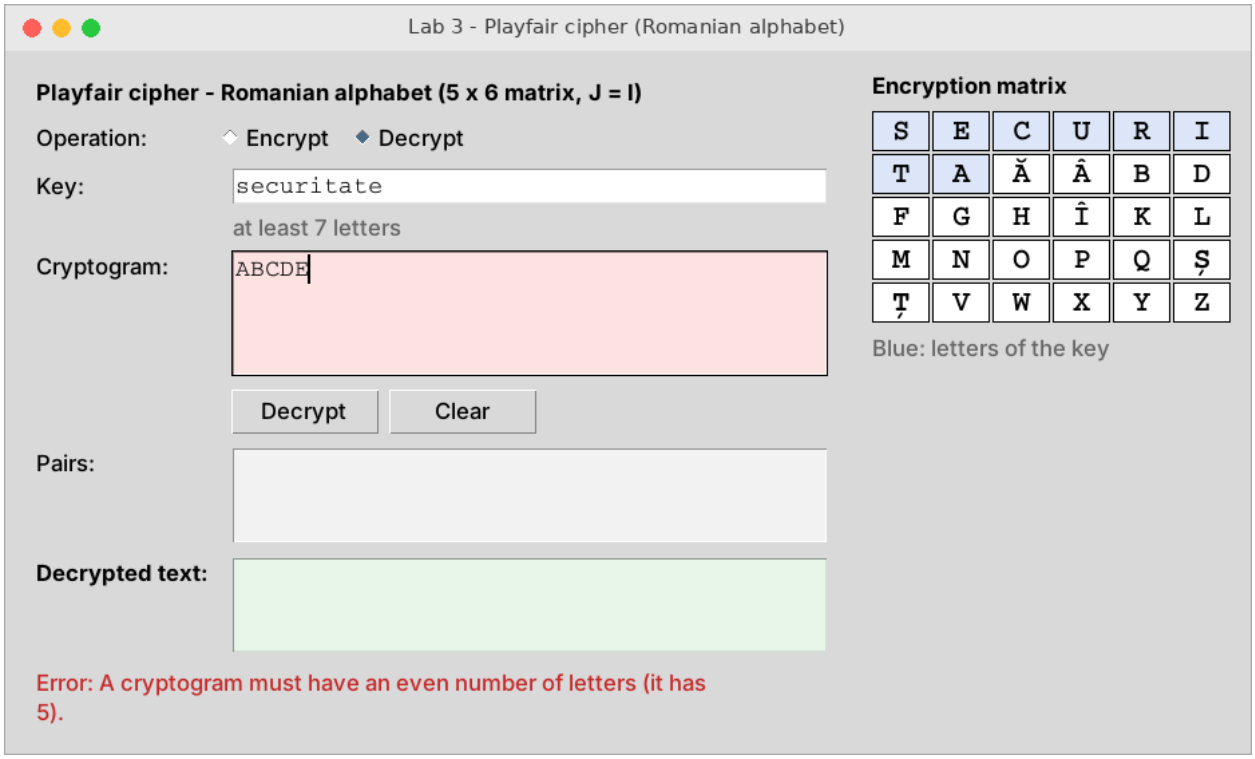


Figure 3.6 – A cryptogram with an odd number of letters is rejected

4. Conclusions

The Playfair cipher encrypts pairs of letters, so the same plaintext letter can be encrypted in different ways depending on the letter it is paired with. This hides the simple letter frequencies that

make monoalphabetic ciphers easy to break. However, the cipher is still a substitution of digrams: the same pair is always encrypted into the same pair, a pair and its reverse give reversed results (AB → XY means BA → YX), and a letter is never encrypted into itself in the rectangle case. With enough ciphertext, the frequencies of digrams can be used to break it.

Adapting the algorithm to Romanian required a different matrix. The 31 letters do not fit into a 5 × 5 matrix, so a 5 × 6 matrix was used and the rare letter J was replaced by I, exactly as in the classic version of the algorithm. X, a very rare letter in Romanian, works well as a filler.

During this work I learned how to build the Playfair matrix from a key, how to prepare the text so that no pair has two equal letters, and how to implement the three encryption rules and their inverses with modular arithmetic on the rows and columns of the matrix.

5. Source code

The full source code is available on GitHub: https://github.com/Afyze/cs-2026-AugustinCarazanu/tree/main/lab3